

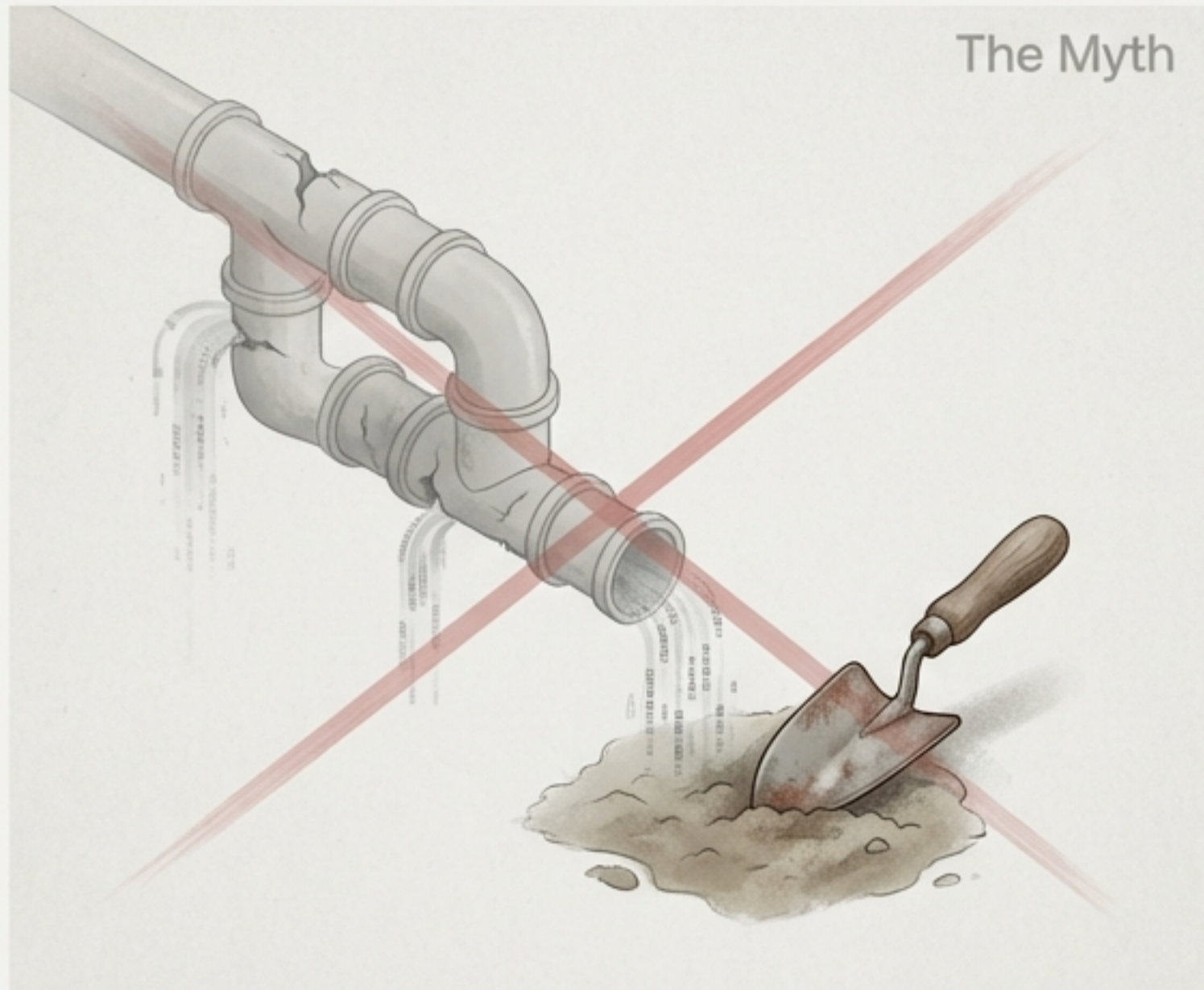
The Future of Developers

More paths in, not out.

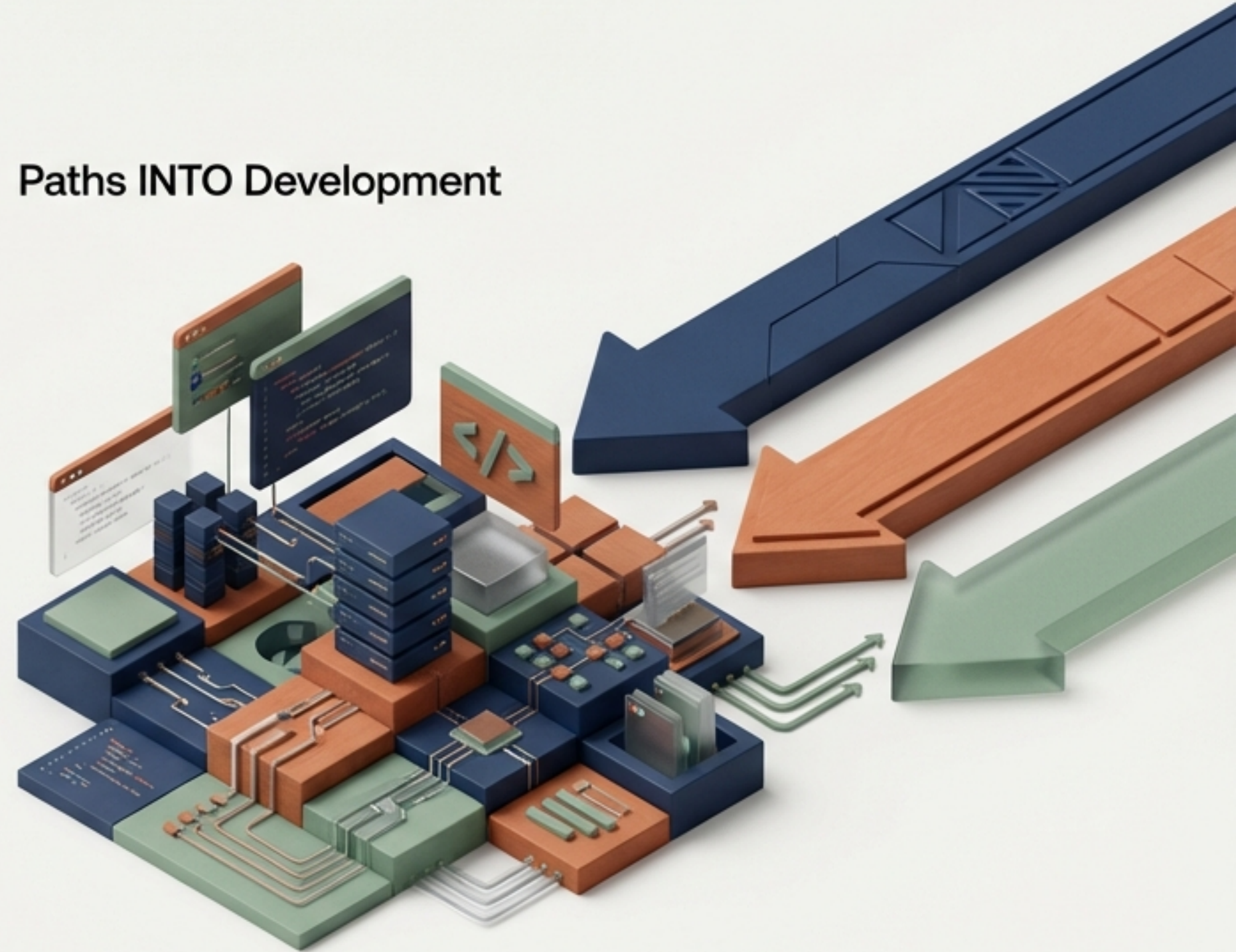


The Great Misconception

A prevailing industry anxiety suggests AI and automation will push software engineers out of the profession—relegating them to unrelated careers like gardening. The reality is exactly backwards.

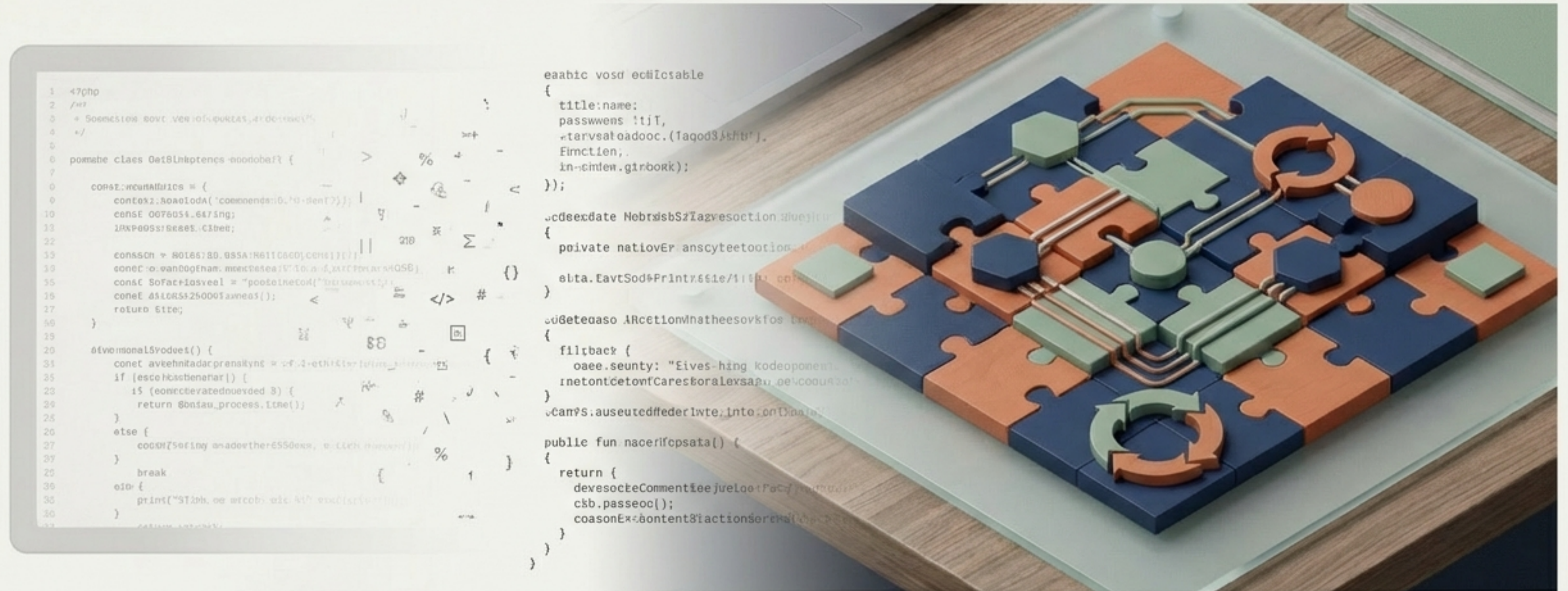


Paths INTO Development



Redefining the Craft

Development was never about syntax or frameworks. At its core, it is about solving problems. It requires breaking complex systems into small pieces, iterating, refactoring, and mapping evolution.



The Vibe Coding Explosion

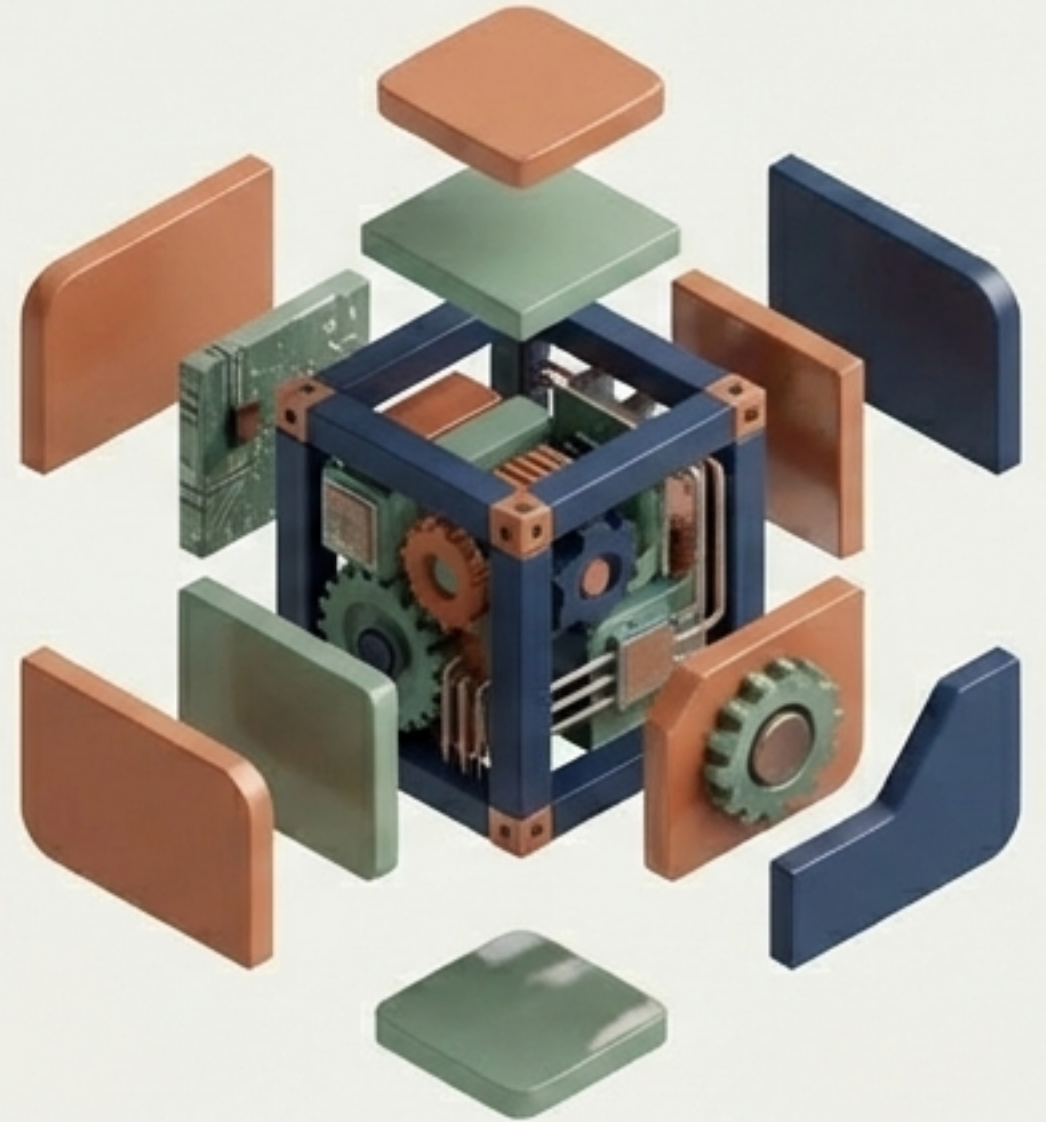
Management has been “vibe coding” their entire careers with intent and vision; it was just painfully slow to iterate. Today, AI allows business owners to create working prototypes instantly. Excel was the original vibe coding tool—we are simply seeing a massive upgrade in rapid iteration.



The Origin



The Evolution



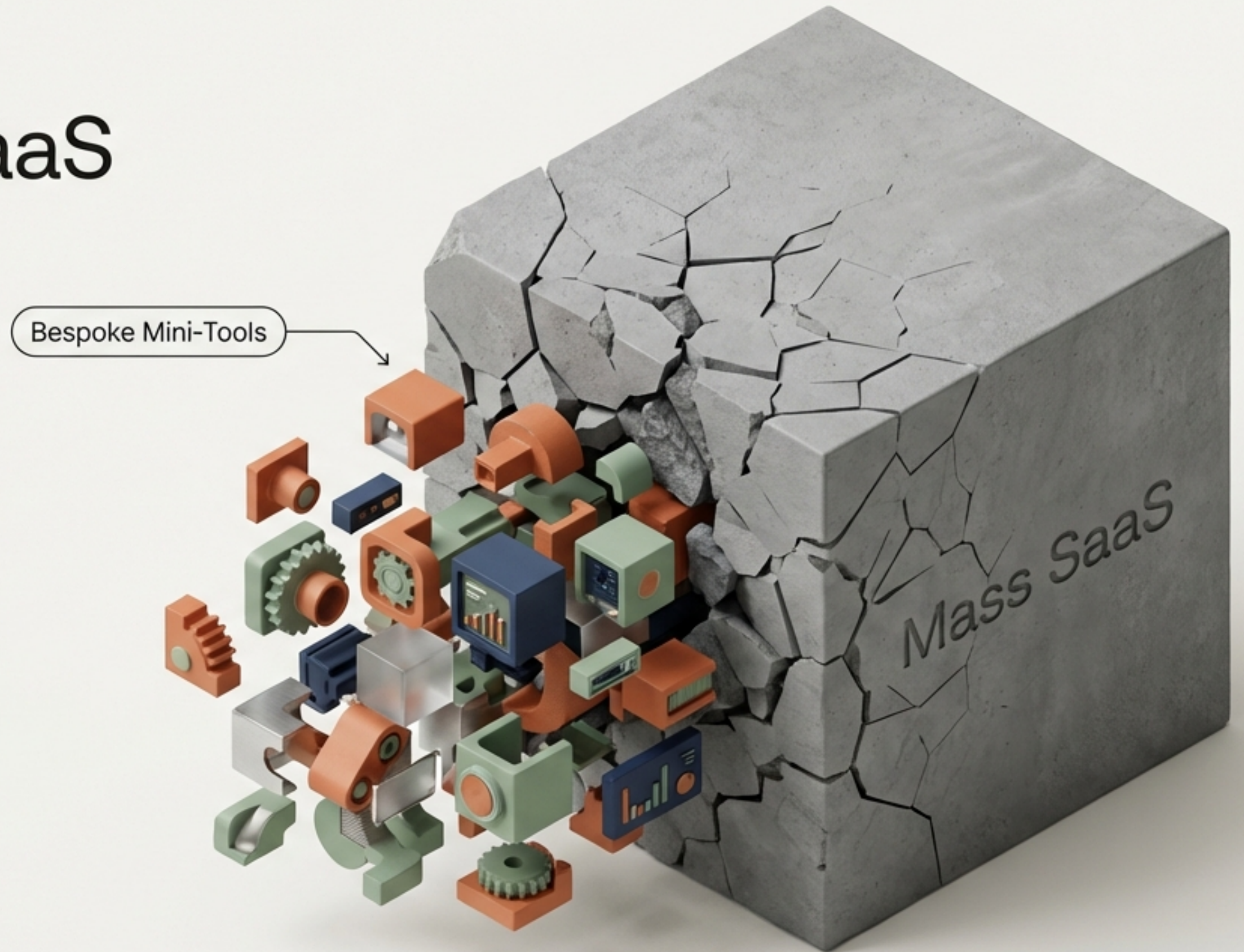
Today's Rapid Iteration

The End of Mass-Market SaaS

Rapid iteration means the era of one-size-fits-all mass-market software is ending.

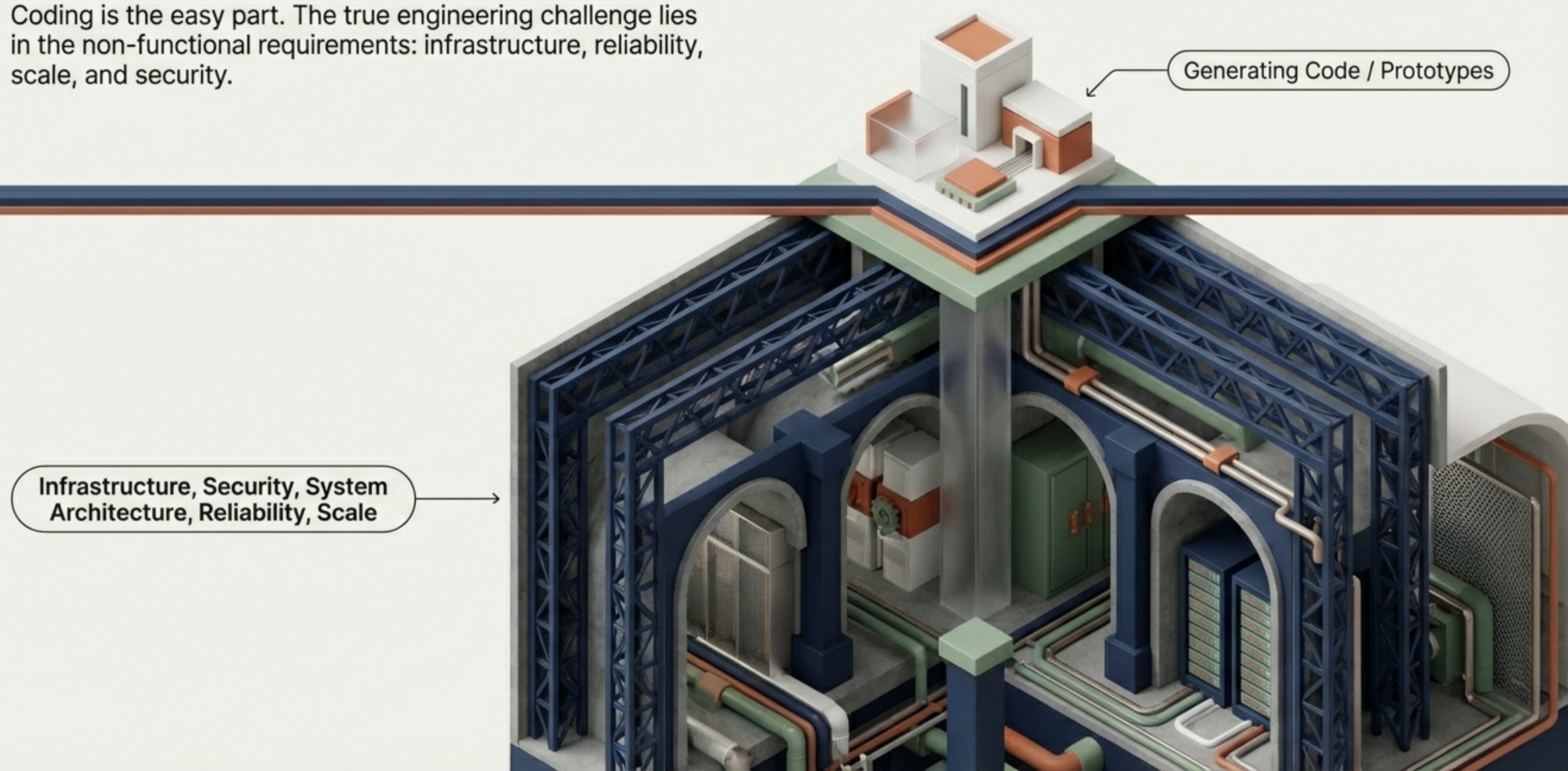
Businesses are abandoning clunky applications that solve 80% of a problem but make the remaining 20% excruciating.

The future belongs to bespoke, hyper-local mini-tools.



The Hidden Truth of Software

Coding is the easy part. The true engineering challenge lies in the non-functional requirements: infrastructure, reliability, scale, and security.



The Handover Problem

Vibe coding gets you from zero to prototype. Moving from prototype to production-grade requires deep engineering craft. Even companies like Anthropic, surrounded by AI-coded developers, face systems going up and down. Scaling robust systems remains incredibly hard.

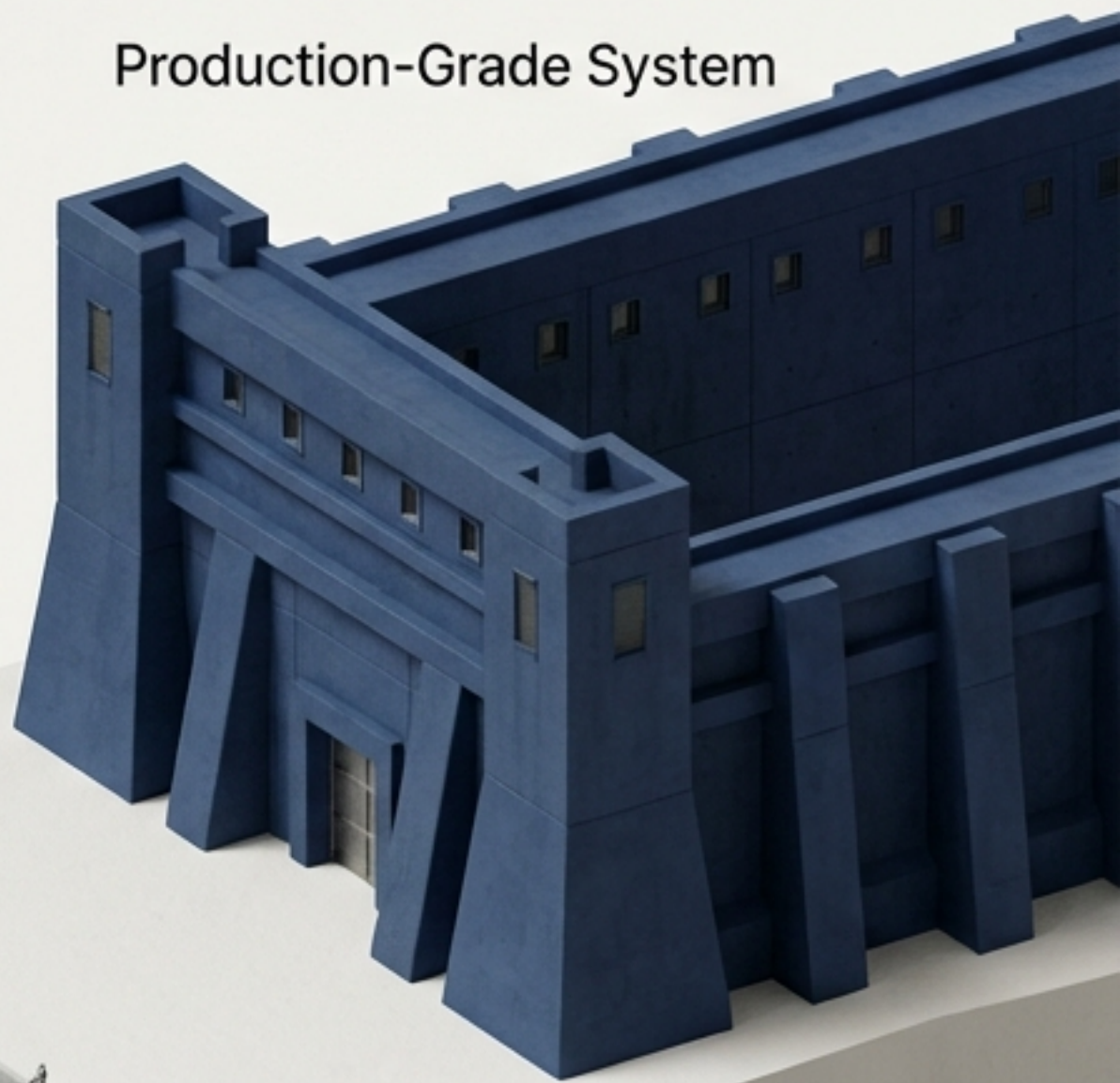
AI-Generated Prototype



The Handover Problem

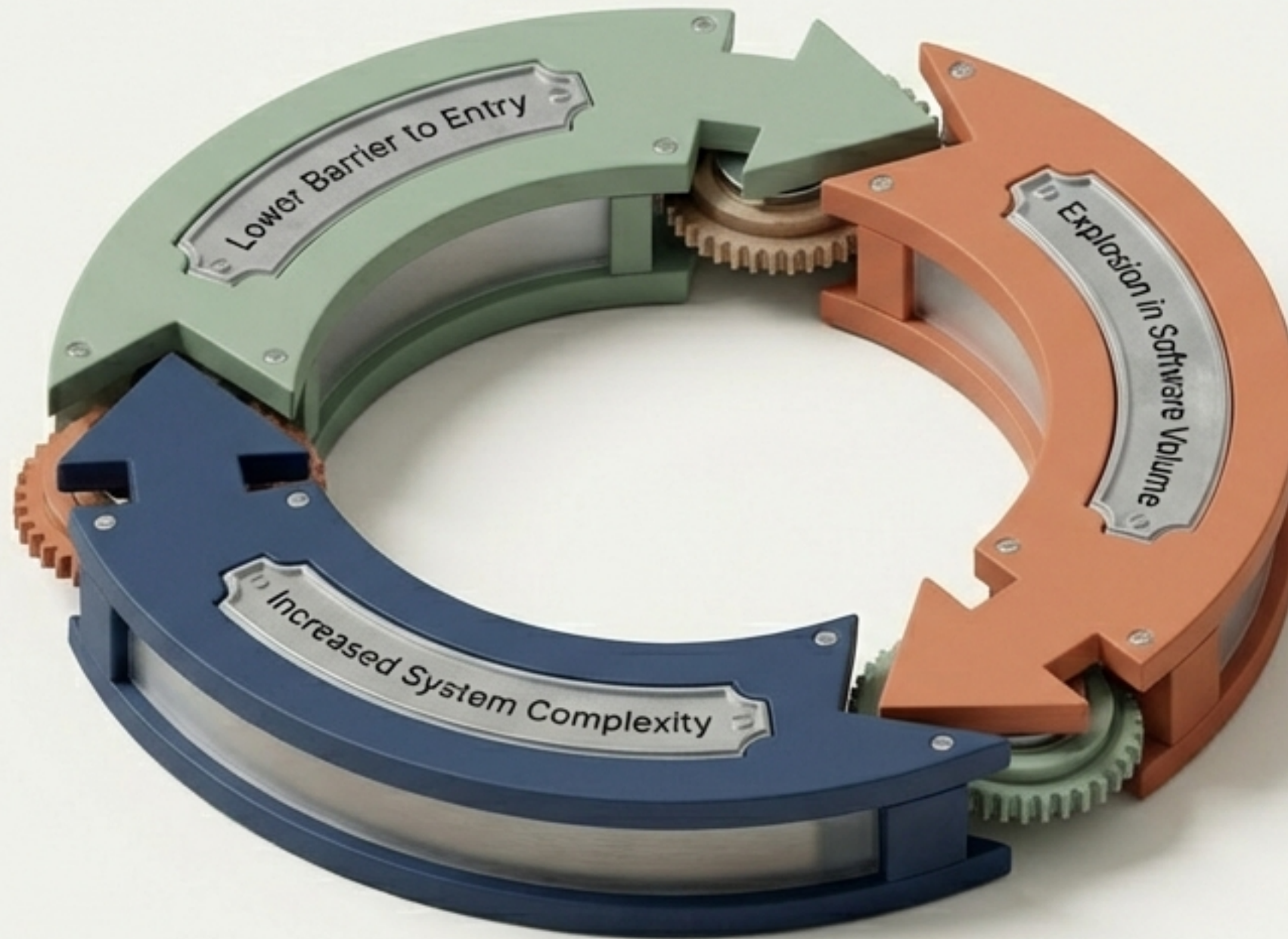


Production-Grade System



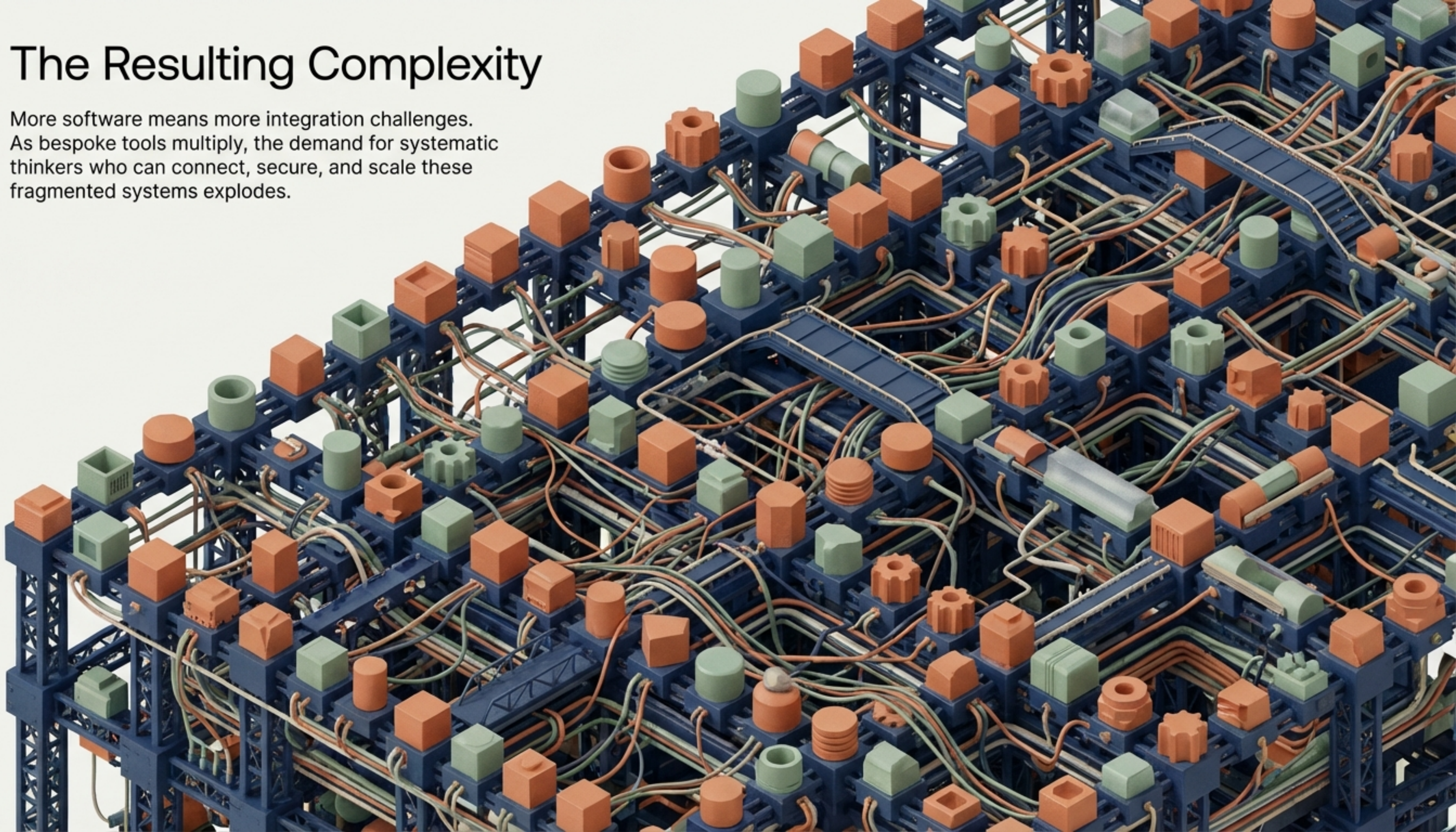
Jevons Paradox in Software

When a resource becomes more accessible, demand doesn't shrink—it skyrockets. Lowering the barrier to entry for creating software massively increases the total volume of software being produced.



The Resulting Complexity

More software means more integration challenges. As bespoke tools multiply, the demand for systematic thinkers who can connect, secure, and scale these fragmented systems explodes.



Expanding the Talent Pipeline

The next generation of developers will not come exclusively from traditional computer science. The industry risks descending into dogma if it fails to consider entirely **different solutions**. We must look outside.



The Interdisciplinary Developer

Non-traditional backgrounds bring critical new capabilities to software development:



Philosophy

First-principles reasoning



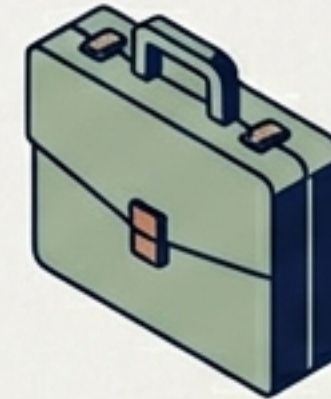
Design

Human-centred problem solving



History

Pattern recognition and system evolution



Business

Mapping process to real value

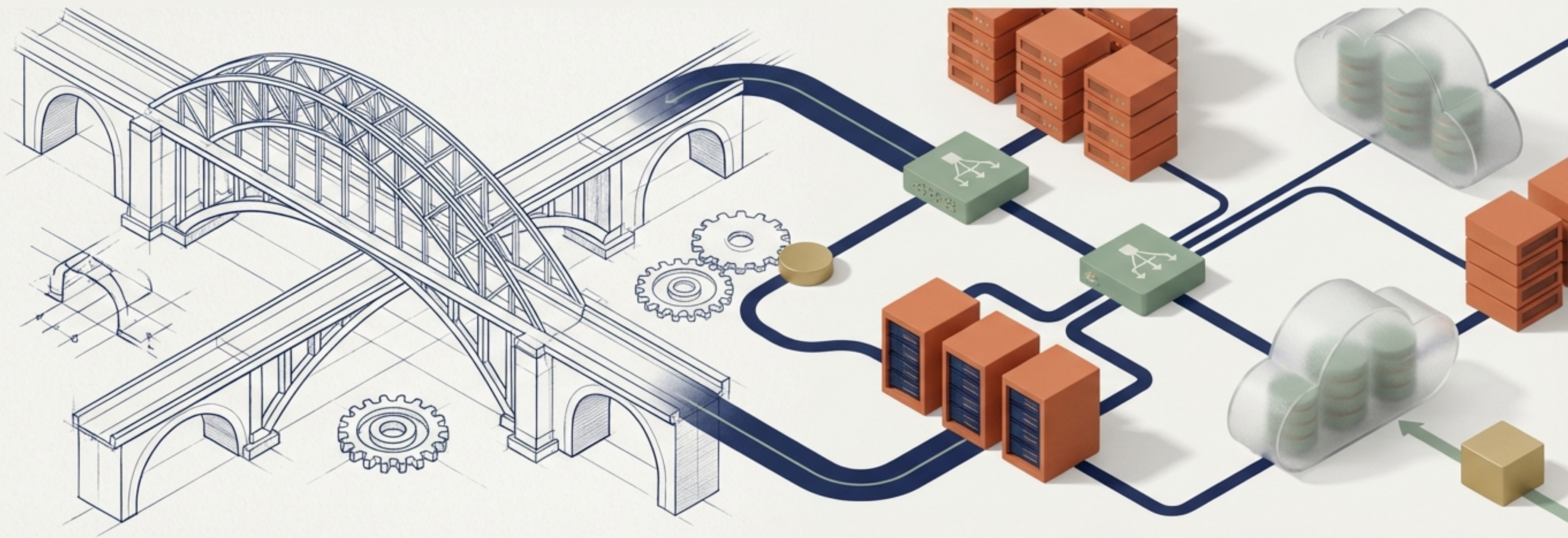


Humanities

Perspective-taking and communication

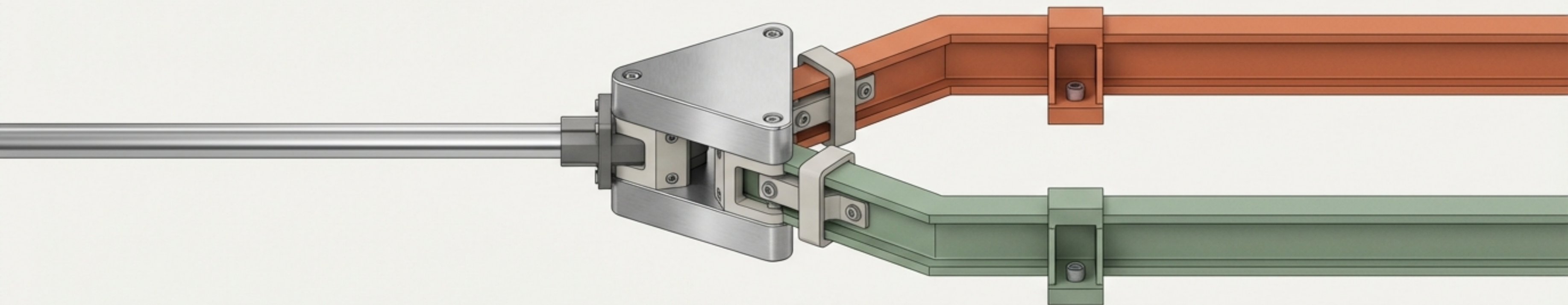
Bringing Back the Hard Sciences

As software becomes critical infrastructure, it requires disciplined engineering practices. Professionals from mechanical engineering, civil engineering, and chemistry will supply the rigorous mindset required to build systems that do not fail.



The Great Split

The monolithic developer role is evolving into two distinct, complementary paths. Both are essential. Both are developers. They simply operate at different points in the value chain.



Two Emerging Archetypes



The Business Developer

Bridges the gap between organisational needs and builds. Over-indexes on user empathy, process understanding, and shipping.



The Craft Developer

Ensures what is released works at scale. Focuses on code review, rock-solid quality, reliability, and security.

The New Standard for Teams

In the past, embedding engineers directly into every business unit was a rare advantage. Today, it is mandatory. Every single team in a business will need these embedded professionals. The future isn't developers leaving the profession; it's the doors opening wider than ever.

